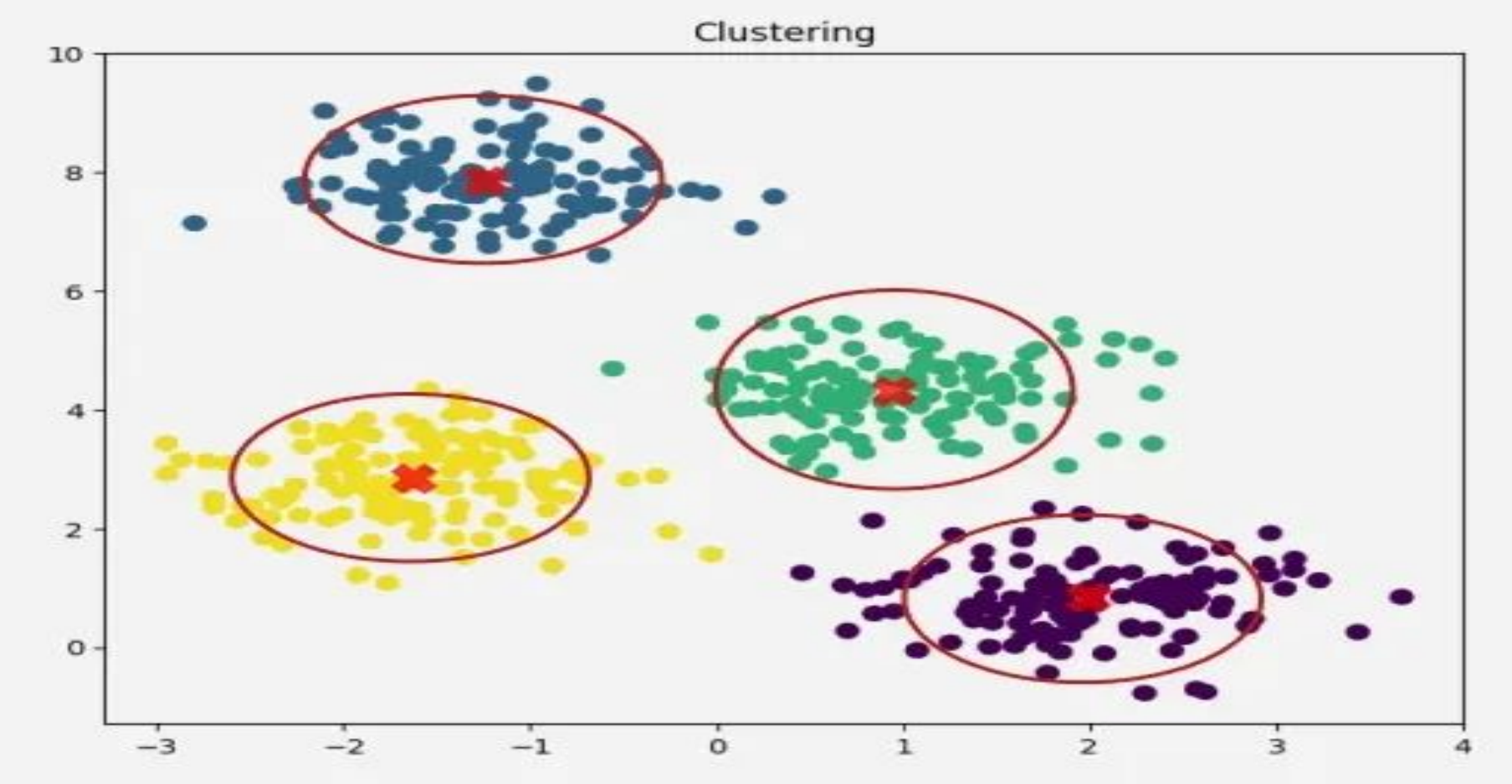
Clustering

Clustering is an unsupervised machine learning technique that groups similar data points together into clusters based on their characteristics, without using any labeled data. The objective is to ensure that data points within the same cluster are more similar to each other than to those in different clusters, enabling the discovery of natural groupings and hidden patterns in complex datasets.

- **Goal**: Discover the natural grouping or structure in unlabeled data without predefined categories.
- **How**: Data points are assigned to clusters based on similarity or distance measures.
- **Similarity Measures**: Can include Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.
- **Output**: Each group is assigned a cluster ID, representing shared characteristics within the cluster.


Clustering
10
8
6
4
2
0
−3
−2
−1
0
1
2
3
4

For example, if we have customer purchase data, clustering can group customers with similar shopping habits. These clusters can then be used for targeted marketing, personalized recommendations or customer segmentation.

Why Use K-Means Clustering?

K-Means is popular in a wide variety of applications due to its simplicity, efficiency and effectiveness. Here's why it is widely used:

1. **Data Segmentation:** One of the most common uses of K-Means is segmenting data into distinct groups. For example, businesses use K-Means to group customers based on behavior, such as purchasing patterns or website interaction.
2. **Image Compression**: K-Means can be used to reduce the complexity of images by grouping similar pixels into clusters, effectively compressing the image. This is useful for image storage and processing.
3. **Anomaly Detection:** K-Means can be applied to detect anomalies or outliers by identifying data points that do not belong to any of the clusters.
4. **Document Clustering:** In natural language processing (NLP), K-Means is used to group similar documents or articles together. It's often used in applications like recommendation systems or news categorization.
5. **Organizing Large Datasets:** When dealing with large datasets, K-Means can help in organizing the data into smaller, more manageable chunks based on similarities, improving the efficiency of data analysis.

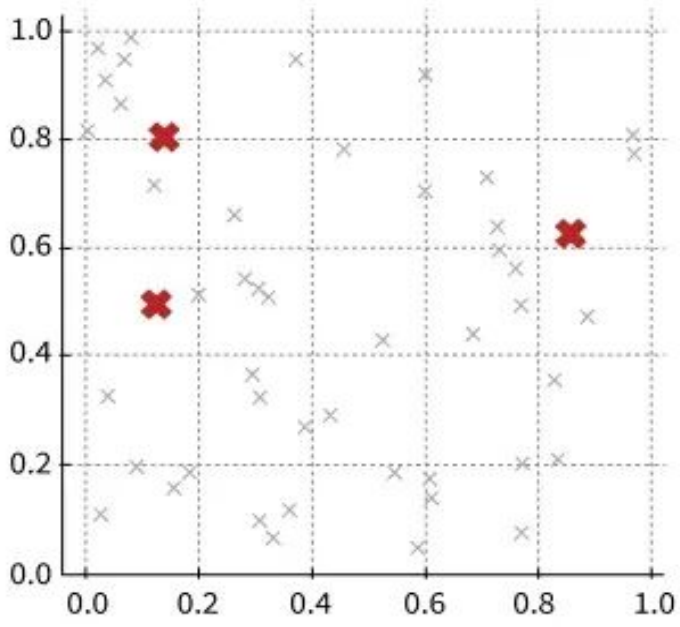
Working of K-Means Clustering

Suppose we are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups. To achieve this we will use the K-means algorithm. "k" represents the number of groups or clusters we want to classify our items into.

Choose Initial Centroids

Centroids are randomly chosen from the data points. These represent the initial cluster centers.

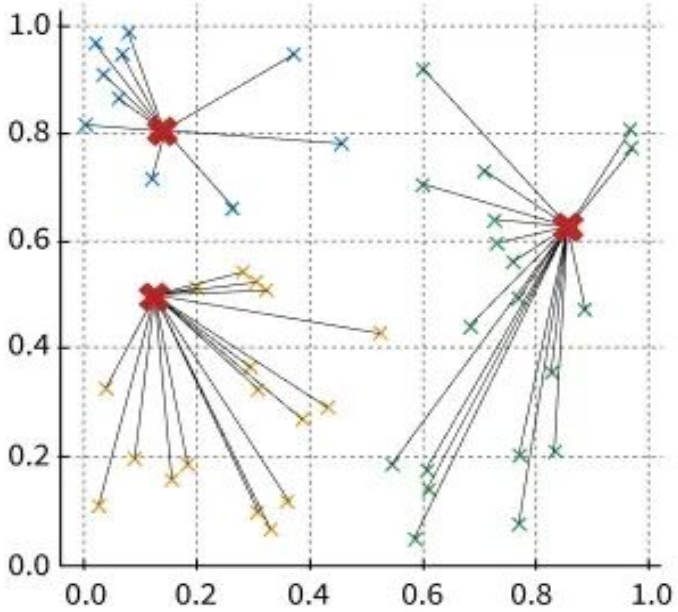
✖ Centroids × Data Points



Assign Points to Nearest Centroid

Each point is assigned to the nearest centroid, forming clusters

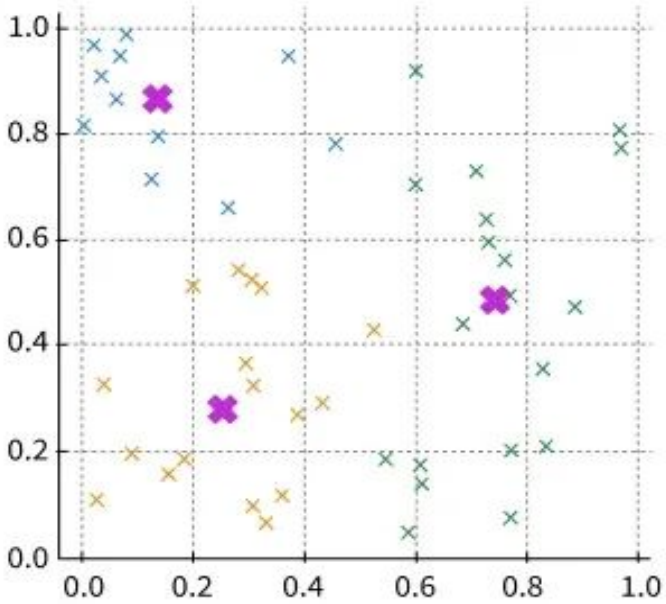
- Centroids
- Cluster 1
- Cluster 2
- Cluster 3



Update Centroids

Centroids are recalculated as the mean of the points in each cluster

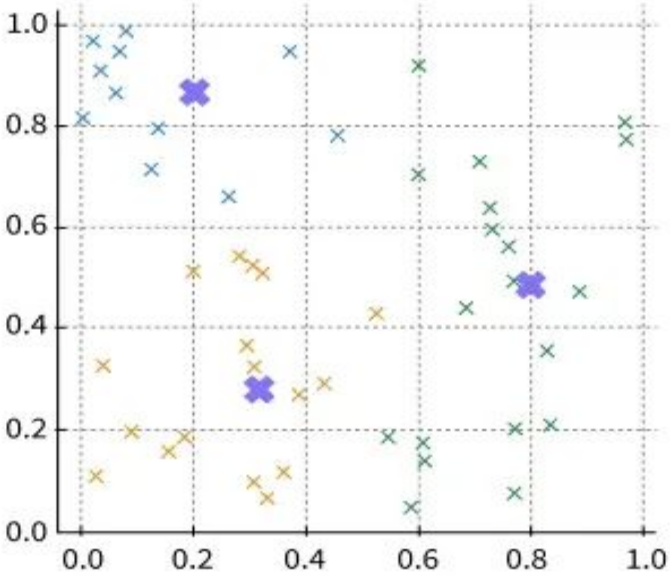
- New Centroids
- Cluster 1
- Cluster 2
- Cluster 3



Repeat Until Convergence

This process repeats until the centroids stabilize and do not move further.

- Final Centroids
- Cluster 1
- Cluster 2
- Cluster 3



The algorithm will categorize the items into “*k*" groups or clusters of similarity. To calculate that similarity we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. **Initialization:** We begin by randomly selecting k cluster centroids.
2. **Assignment Step:** Each data point is assigned to the nearest centroid, forming clusters.
3. **Update Step:** After the assignment, we recalculate the centroid of each cluster by averaging the points within it.
4. **Repeat:** This process repeats until the centroids no longer change or the maximum number of iterations is reached.

The goal is to partition the dataset into

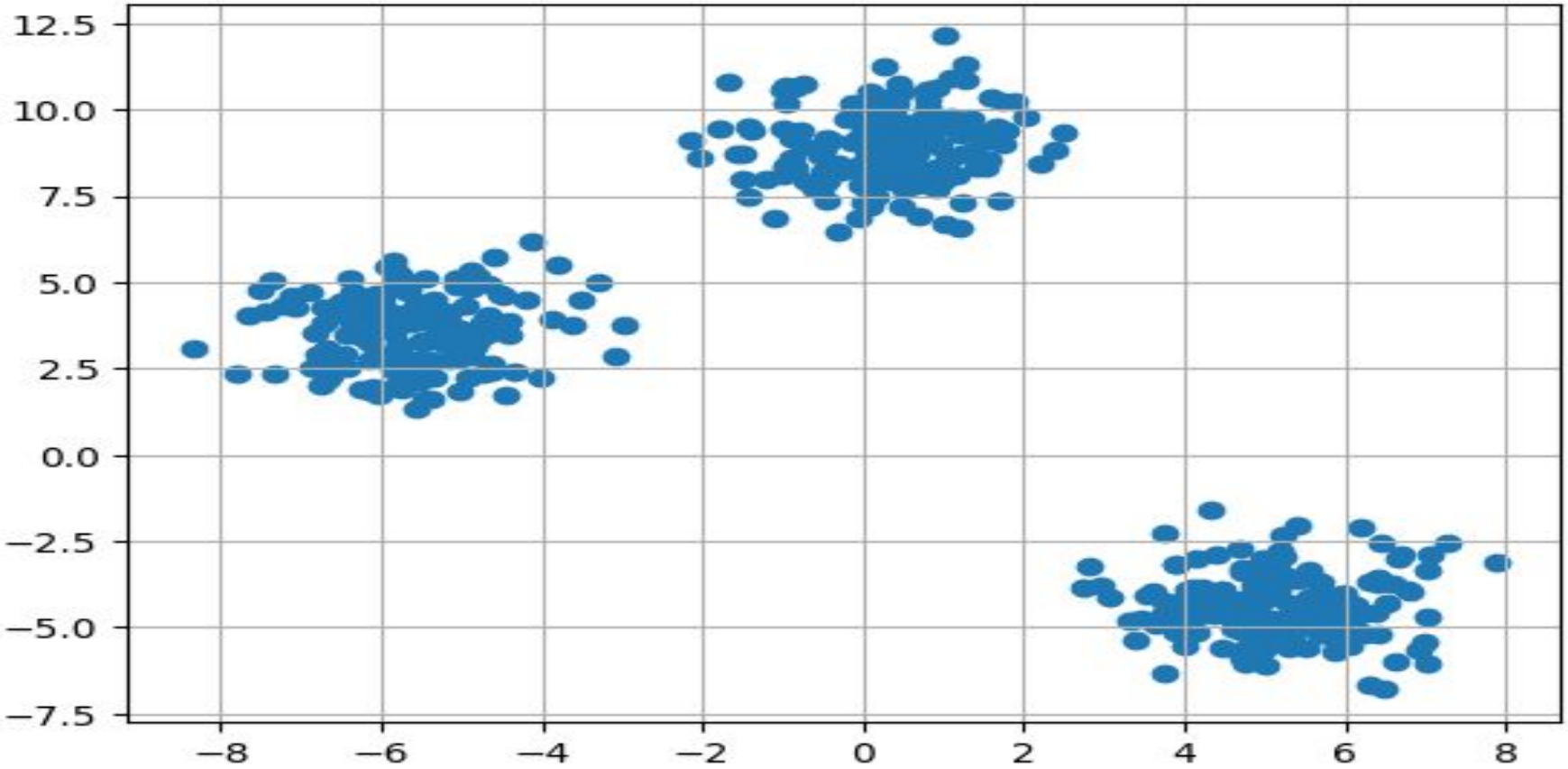
k

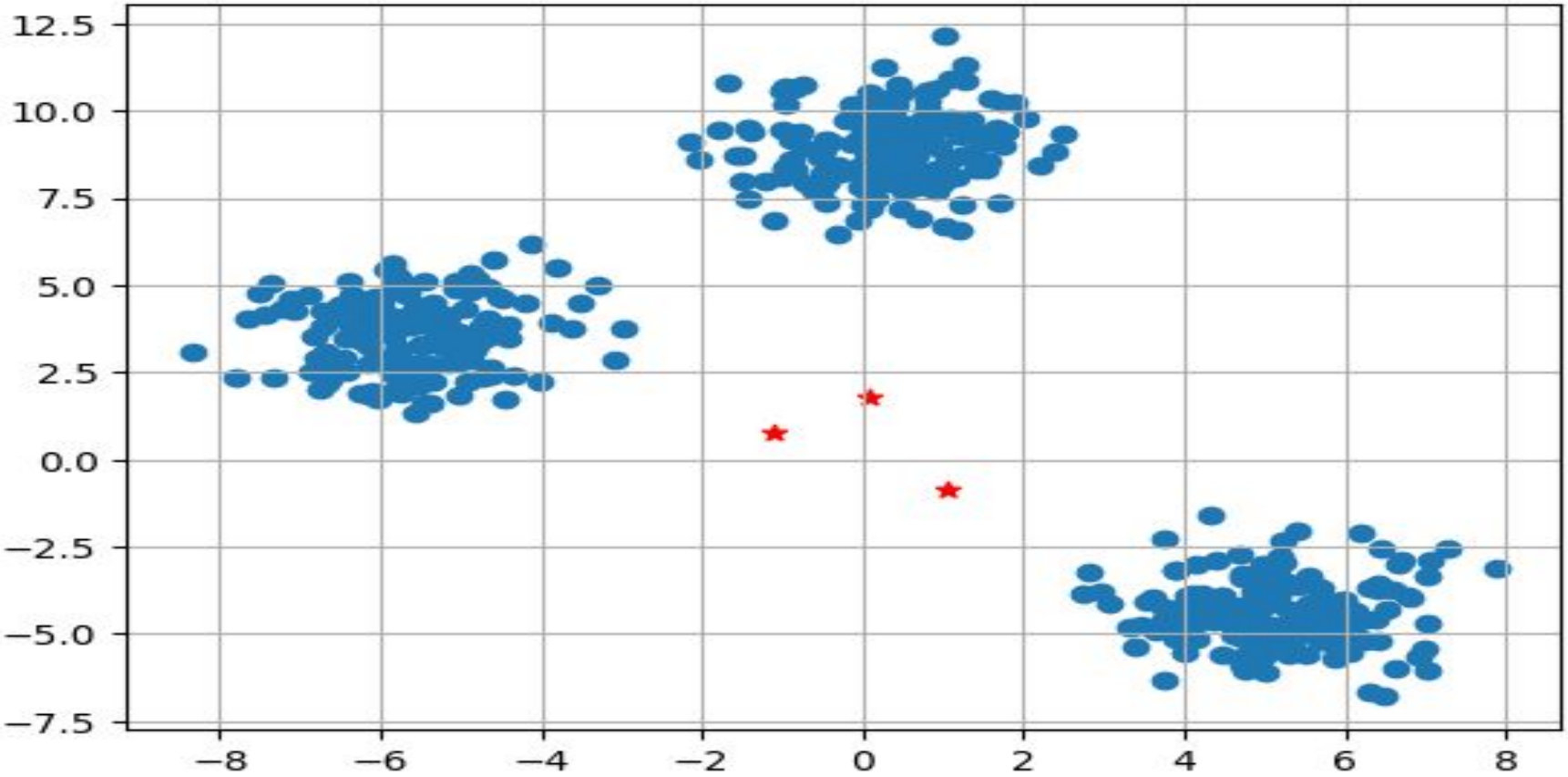
k clusters such that data points within each cluster are more similar to each other than to those in other clusters.

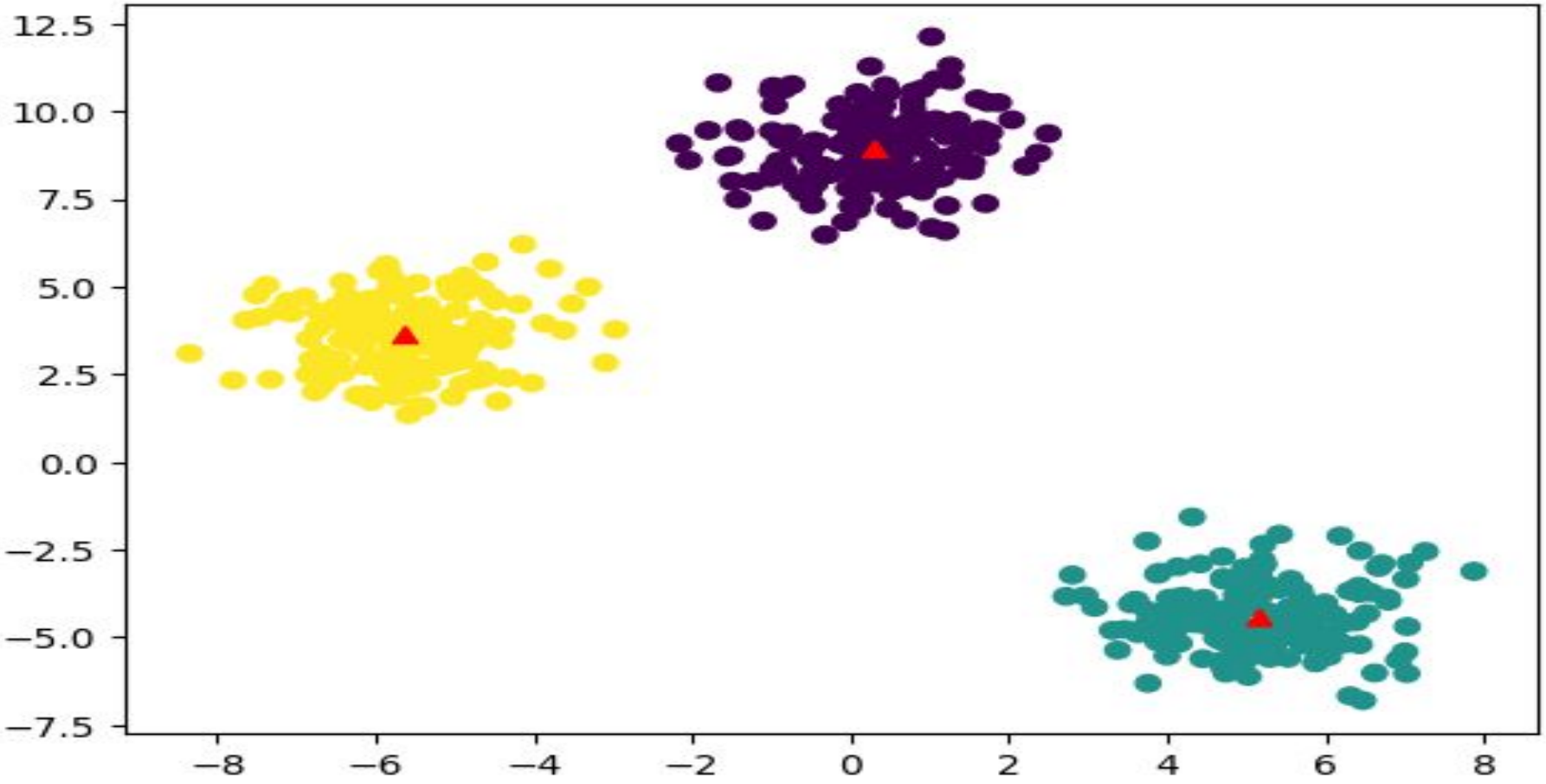
Challenges with K-Means Clustering

K-Means algorithm has the following limitations:

- **Choosing the Right Number of Clusters** (k): One of the biggest challenges is deciding how many clusters to use.
- **Sensitive to Initial Centroids:** The final clusters can vary depending on the initial random placement of centroids.
- **Non-Spherical Clusters:** K-Means assumes that the clusters are spherical and equally sized. This can be a problem when the actual clusters in the data are of different shapes or densities.
- **Outliers**: K-Means is sensitive to outliers, which can distort the centroid and, ultimately, the clusters.

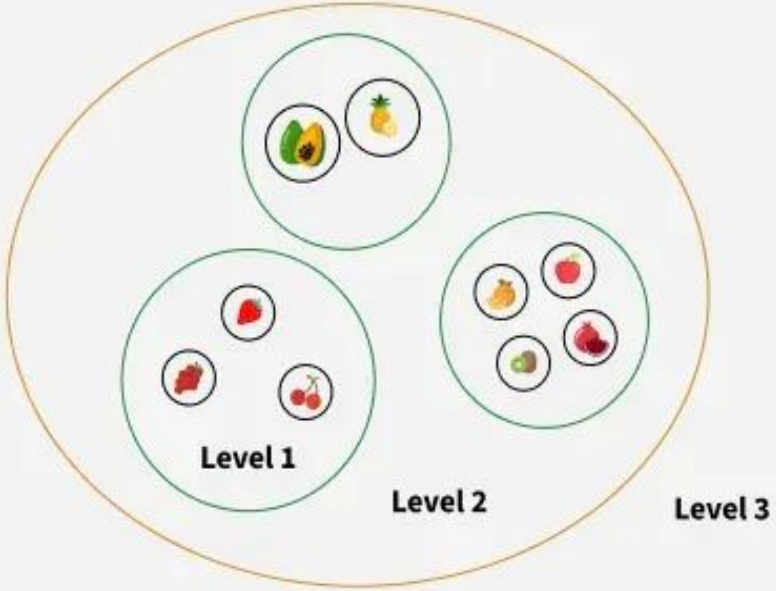

12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−8
−6
−4
−2
0
2
4
6
8


12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−8
−6
−4
−2
0
2
4
6
8


12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−8
−6
−4
−2
0
2
4
6
8

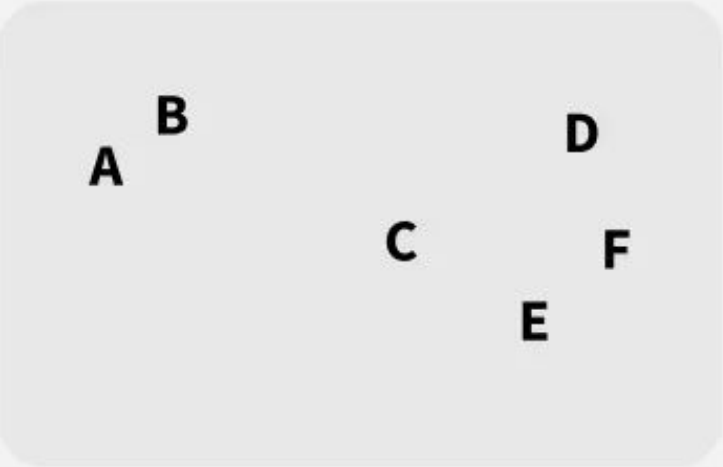
What is Hierarchical Clustering

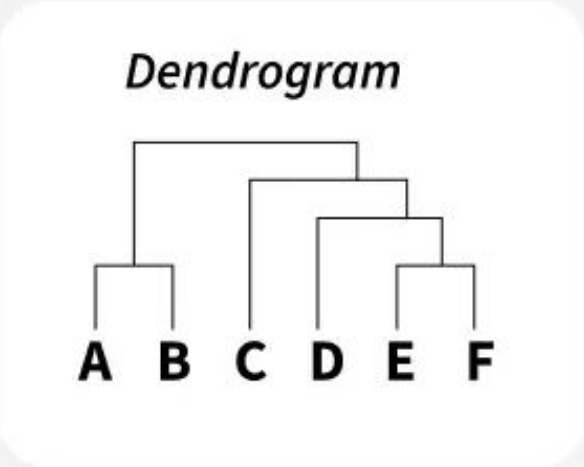
Hierarchical clustering is an unsupervised machine learning algorithm that groups data into a tree of nested clusters

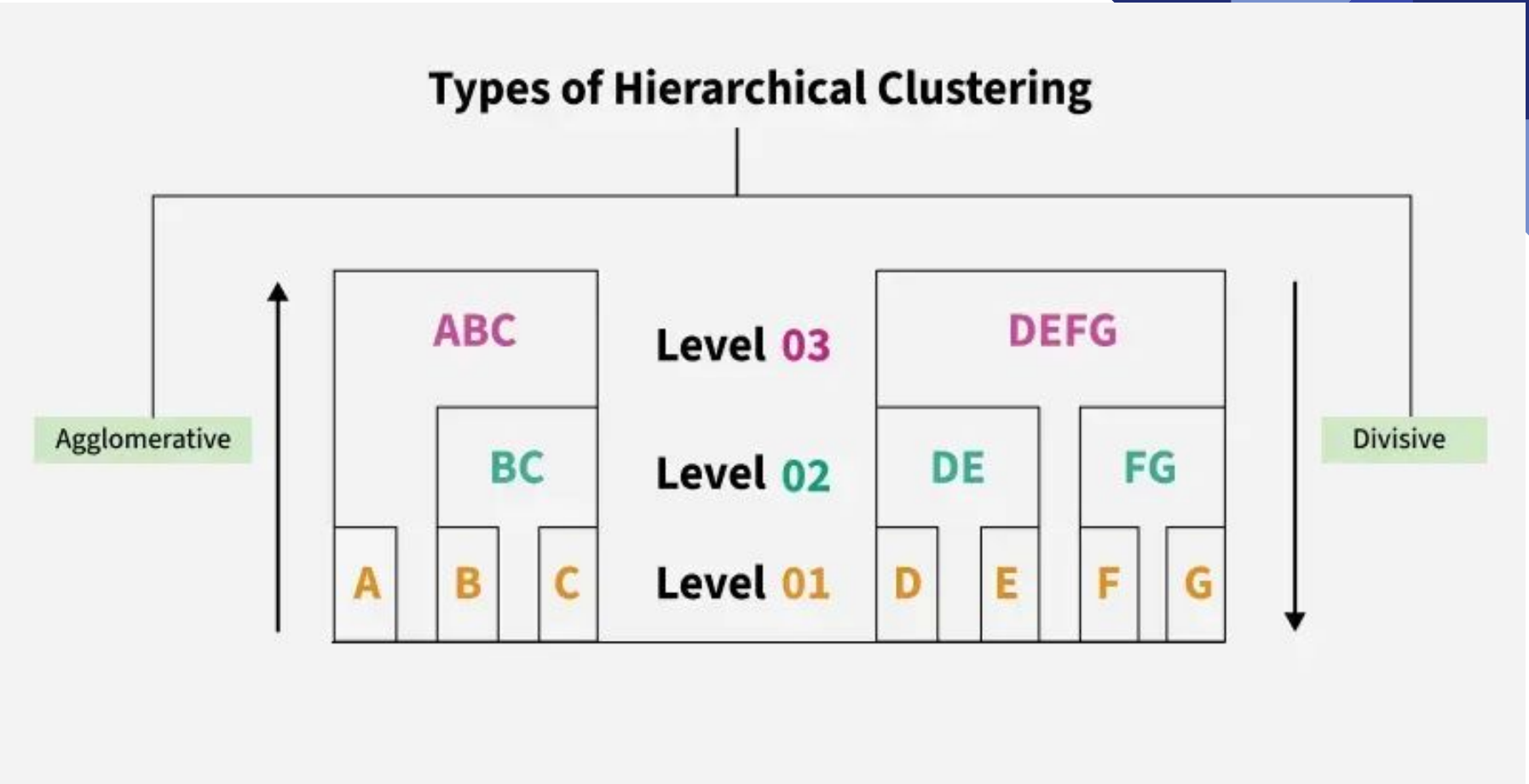


What is a Dendrogram?

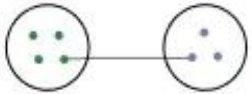
A **dendrogram** is a tree that shows how clusters are merged step-by-step.
We **cut** the dendrogram at a certain height to form final clusters.



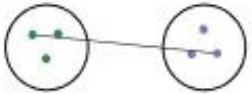



Types of Hierarchical Clustering
Agglomerative
Divisive
ABC
BC
A
B
C
Level 03
Level 02
Level 01
DEFG
DE
FG
D
E
F
G

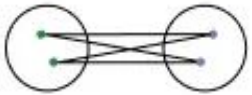
Distance Metrics in Hierarchical Clustering



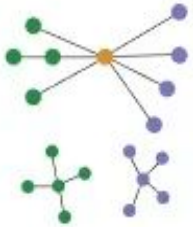
Minimum Distance



Maximum Distance



Group Average



Ward's Method

Applications of Hierarchical Clustering

- Portfolio Construction & Optimisation
- Sector & Industry Analysis
- Pairs Trading Strategies
- Market Structure Understanding
- Behavioral Finance Analysis
- Algorithmic Trading
- Dynamic Asset Allocation

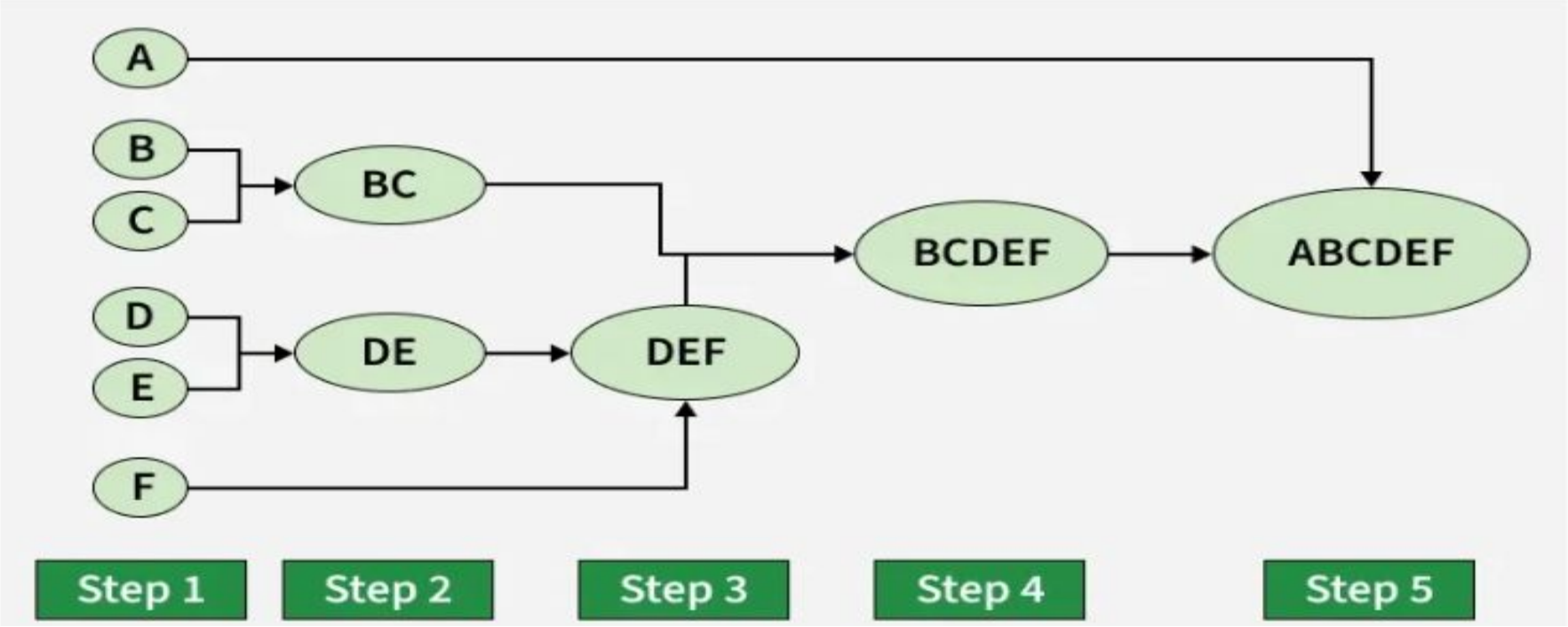
Types of Hierarchical Clustering

Now we understand the basics of hierarchical clustering. There are two main types of hierarchical clustering.

1. Agglomerative Clustering
2. Divisive clustering

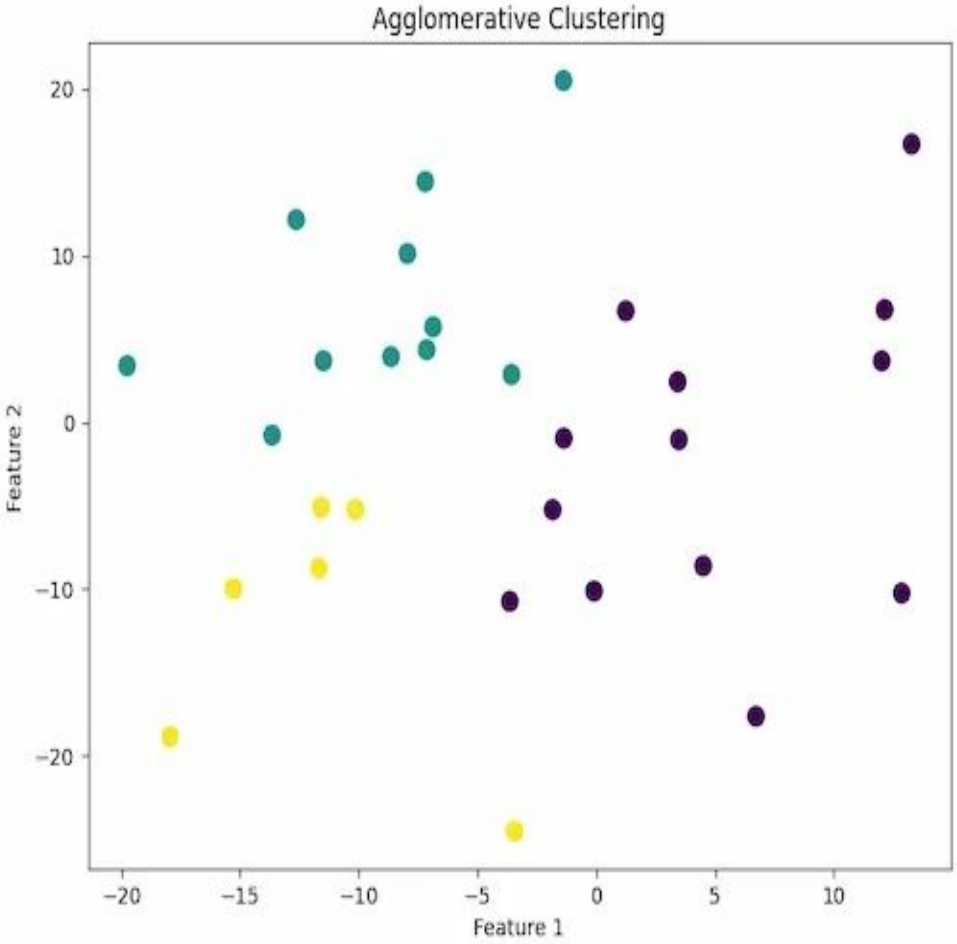
1. Hierarchical Agglomerative Clustering

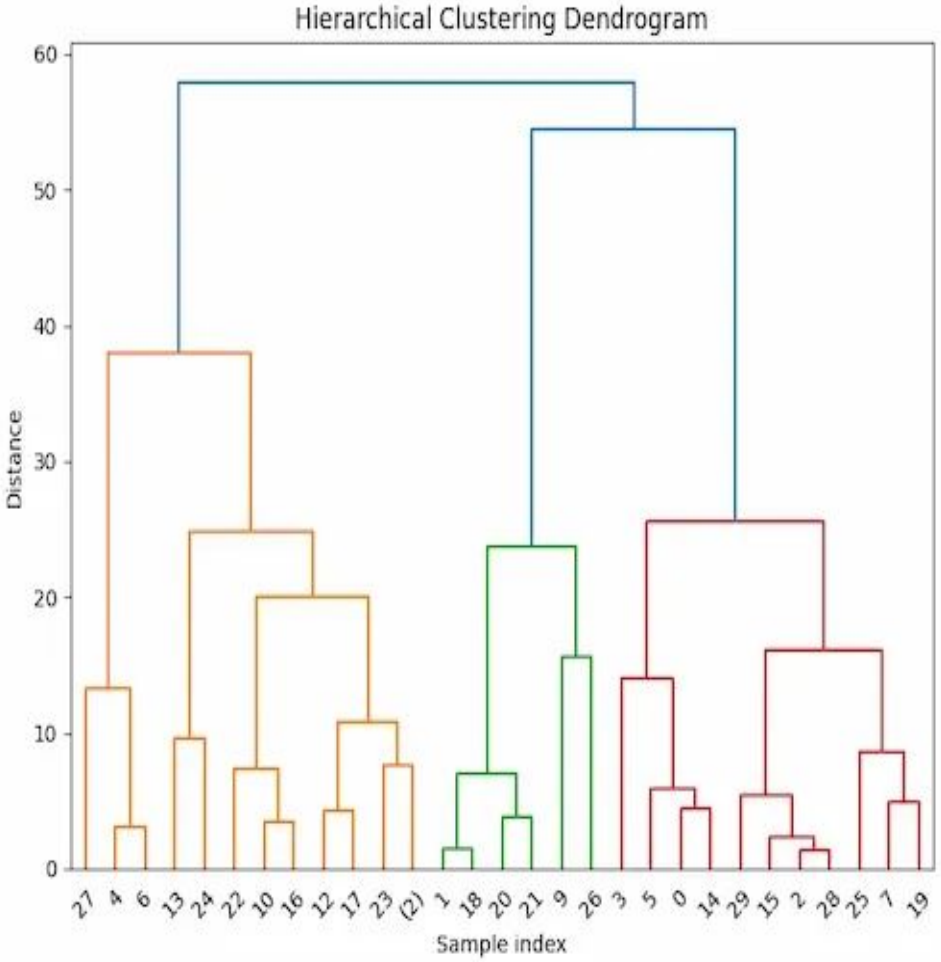
It is also known as the bottom-up approach or hierarchical agglomerative clustering (HAC). Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerate pairs of clusters until all clusters have been merged into a single cluster that contains all data.



Workflow for Hierarchical Agglomerative clustering

1. **Start with individual points:** Each data point is its own cluster. For example if we have 5 data points we start with 5 clusters each containing just one data point.
2. **Calculate distances between clusters**: Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters**: Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix**: After merging we now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
5. **Repeat steps 3 and 4**: Keep merging the closest clusters and updating the distance matrix until we have only one cluster left.
6. **Create a dendrogram**: As the process continues we can visualize the merging of clusters using a tree-like diagram called a dendrogram. It shows the hierarchy of how clusters are merged.


Agglomerative Clustering
Feature 2
Feature 1
20
10
0
−10
−20
−20
−15
−10
−5
0
5
10

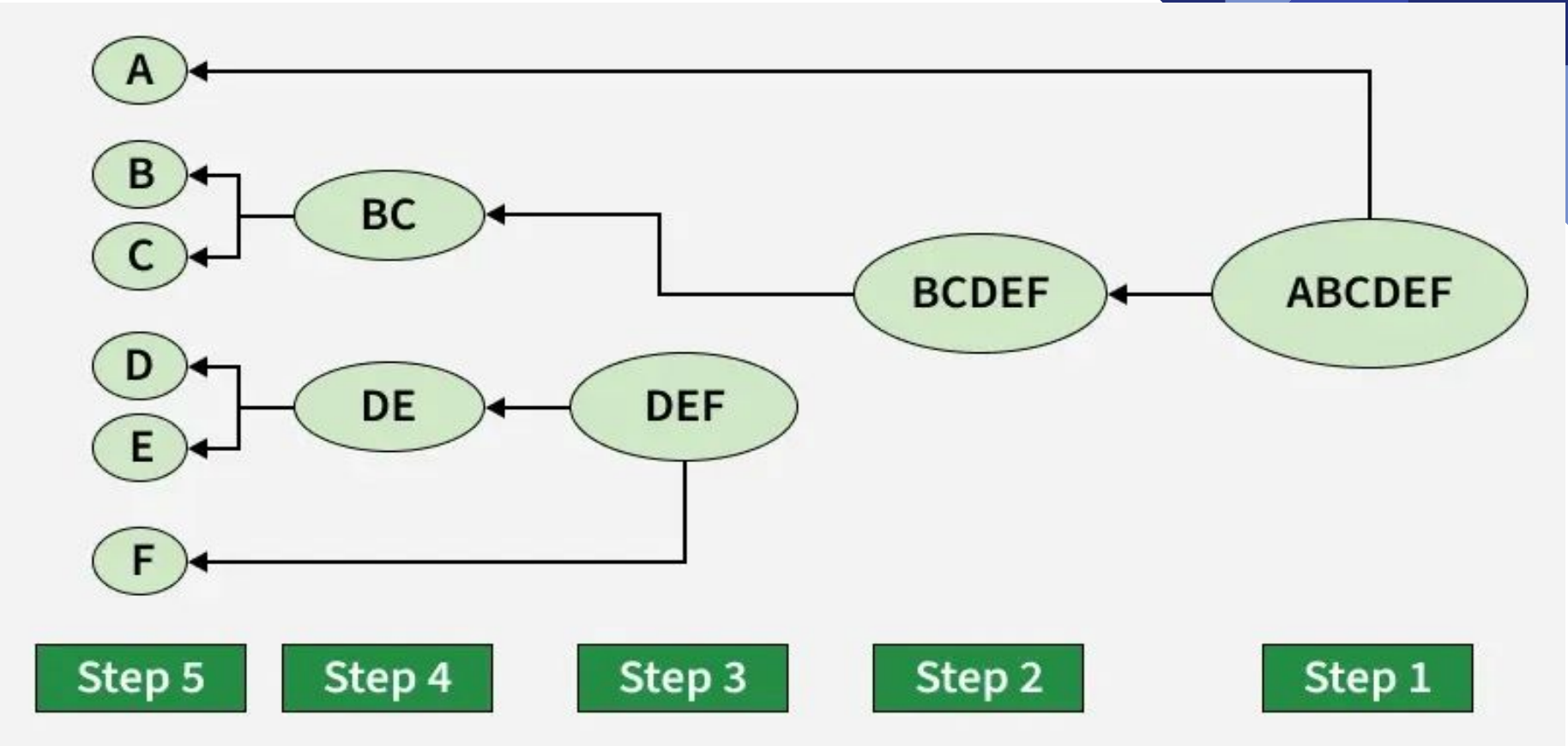

Hierarchical Clustering Dendrogram
Distance
Sample index
60
50
40
30
20
10
0
27 4 6 13 24 22 10 16 12 17 23 (2) 1 18 20 21 9 26 3 5 0 14 29 15 2 28 25 7 19

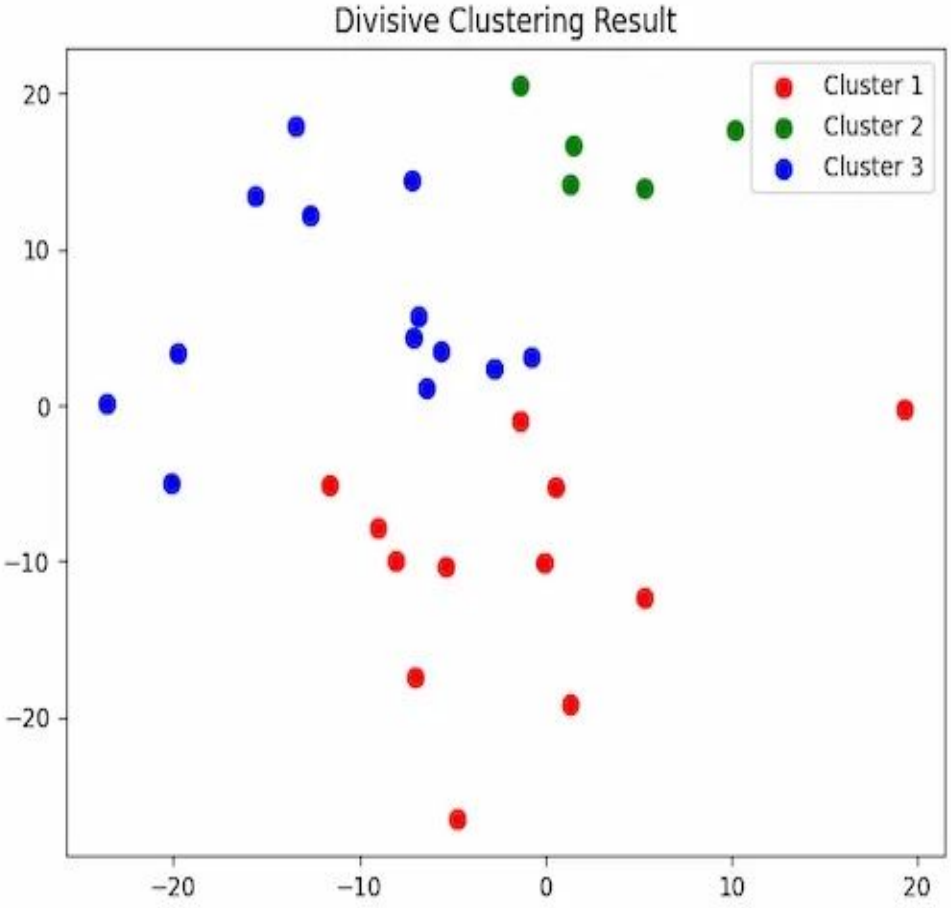
2. Hierarchical Divisive clustering

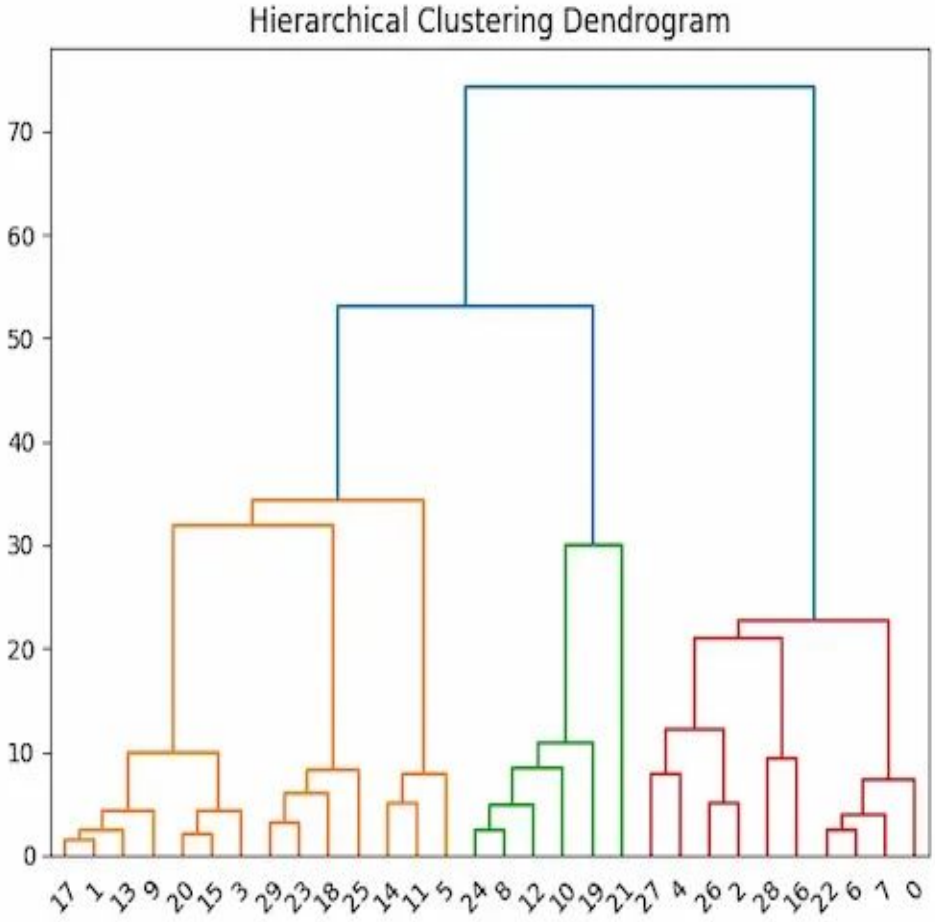
Divisive clustering is also known as a top-down approach. Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been split into singleton clusters.

Workflow for Hierarchical Divisive clustering :

1. **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
2. **Split the cluster**: Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
3. **Repeat the process**: For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.
4. **Stop when each data point is in its own cluster**: Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.


A
B
C
BC
BCDEF
ABCDEF
D
E
DE
DEF
F
Step 5
Step 4
Step 3
Step 2
Step 1

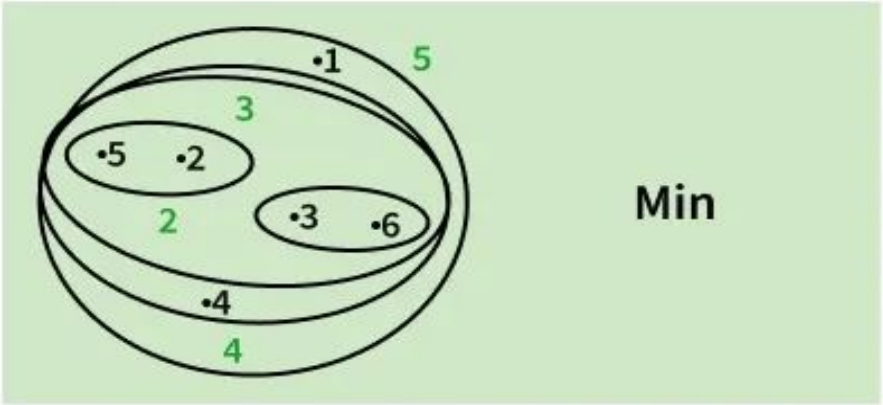

Divisive Clustering Result
Cluster 1
Cluster 2
Cluster 3
20
10
0
−10
−20
−20
−10
0
10
20


Hierarchical Clustering Dendrogram
70
60
50
40
30
20
10
0
17 1 13 9 20 15 3 29 23 18 25 14 11 5 24 8 12 10 19 21 27 4 26 2 28 16 22 6 7 0

Computing Distance Matrix

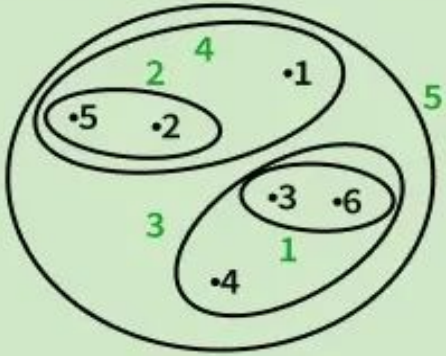
While merging two clusters we check the distance between two every pair of clusters and merge the pair with the least distance/most similarity. But the question is how is that distance determined. There are different ways of defining Inter Cluster distance/similarity. Some of them are:

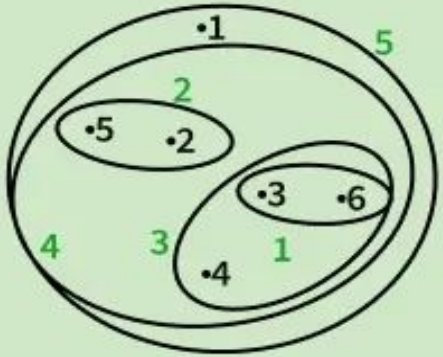
- **Min Distance**: Find the minimum distance between any two points of the cluster.
- **Max Distance:** Find the maximum distance between any two points of the cluster.
- **Group Average**: Find the average distance between every two points of the clusters.
- **Ward's Method**: The similarity of two clusters is based on the increase in squared error when two clusters are merged.



Min

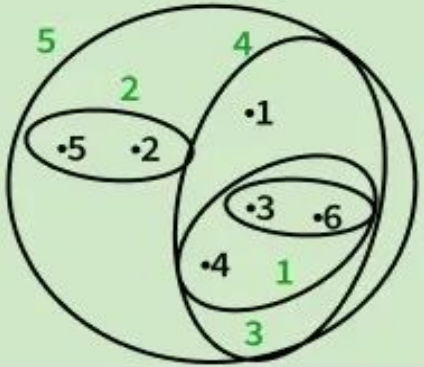
Max





Group Average

Ward's Method

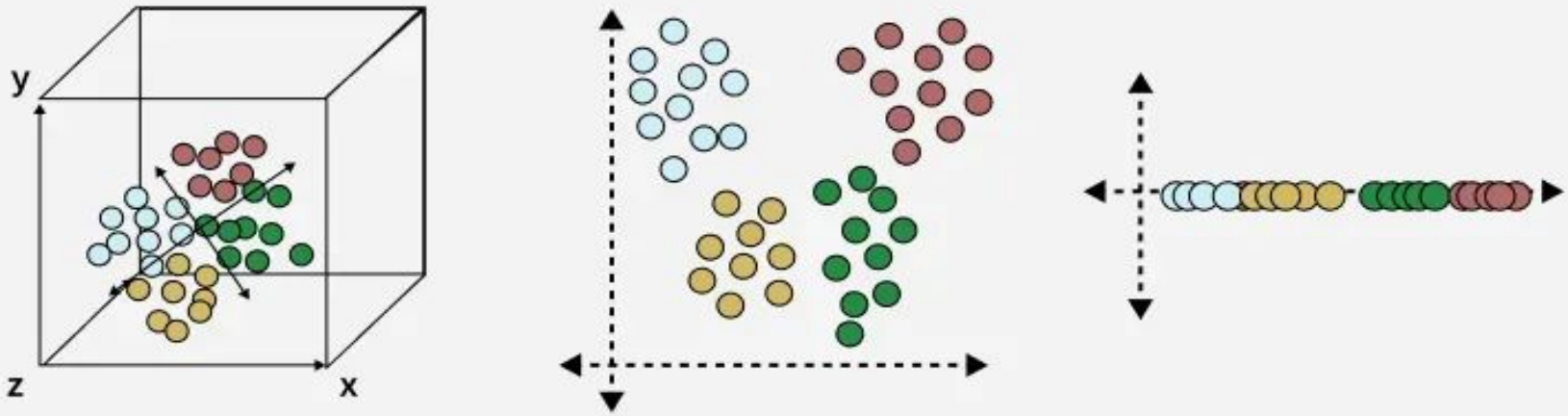


The image compares cluster distance methods:

- Min uses the shortest distance between clusters
- Max uses the longest
- Group Average computes the mean of all pairwise distances
- Ward’s method minimizes the increase in within-cluster variance during merging

What is Dimensionality Reduction

Dimensionality Reduction is the process of reducing the number of input variables (features) in a dataset while preserving as much important information as possible.



Dimensionality Reduction Techniques

Feature selection

- Filter method
- Wrapper method
- Embedded method

Feature extraction

- Principal Component Analysis
- Missing Value Ratio
- Backward Feature Selection
- Forward Feature Selection
- Factor Analysis
- Independent Component Analysis

Avantages and Disadvantages of Dimensionality Reduction

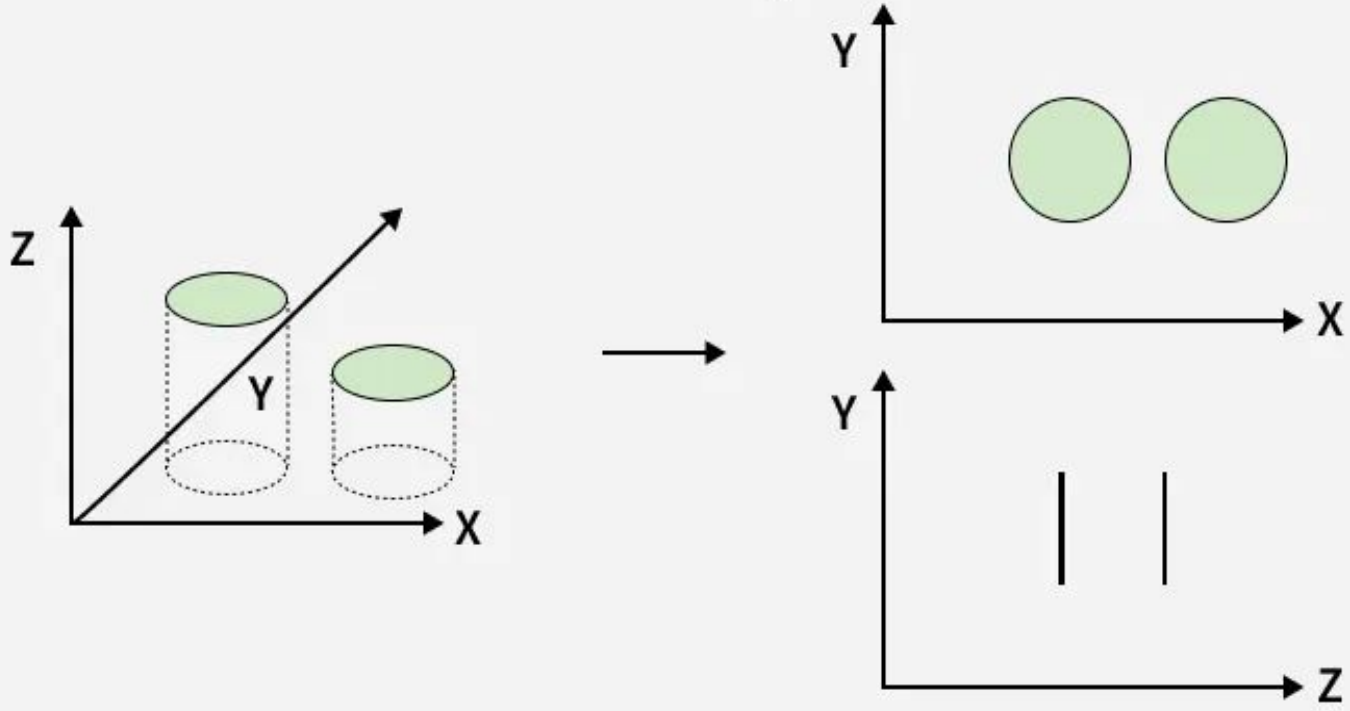
Advantages

- Simplify the pattern representation and the classifiers
- Faster classifier with less memory consumption
- Alleviate curse of dimensionality with limited data sample

Disadvantages

- Loss information
- Increased error in the resulting recognition system

Dimensionality Reduction



- Before Reduction we can see that data exist in 3D (X,Y,Z). It has high redundancy and Z contributes little meaningful information
- On the right after reducing the dimensionality the data is represented in lower-dimensional spaces. The top plot (X-Y) maintains the meaningful structure while the bottom plot (Z-Y) shows that the Z-dimension contributed little useful information.

Dimensionality Reduction Techniques

Dimensionality reduction techniques can be broadly divided into two categories:

1. Feature Selection

Feature selection chooses the most relevant features from the dataset without altering them. It helps remove redundant or irrelevant features, improving model efficiency. Some common methods are:

- Filter methods rank the features based on their relevance to the target variable.
- Wrapper methods use the model performance as the criteria for selecting features.
- Embedded methods combine feature selection with the model training process.

2. Feature Extraction

Feature extraction involves creating new features by combining or transforming the original features. These new features retain most of the dataset's important information in fewer dimensions. Common feature extraction methods are:

1. **Principal Component Analysis (PCA):** Converts correlated variables into uncorrelated principal components hence reducing dimensionality while maintaining as much variance as possible enabling more efficient analysis.
2. **Missing Value Ratio:** Variables with missing data beyond a set threshold are removed, improving dataset reliability.
3. **Backward Feature Elimination**: Starts with all features and removes the least significant ones in each iteration. The process continues until only the most impactful features remain, optimizing model performance.
4. **Forward Feature Selection:** It begins with one feature, adds others incrementally and keeps those improving model performance.
5. **Random Forest**: Random forest uses decision trees to evaluate feature importance, automatically selecting the most relevant features without the need for manual coding, enhancing model accuracy.
6. **Factor Analysis**: Groups variables by correlation and keeps the most relevant ones for further analysis.
7. **Independent Component Analysis (ICA)**: Identifies statistically independent components, ideal for applications like 'blind source separation' where traditional correlation-based methods fall short.

Real World Use Case

Dimensionality reduction plays a important role in many real-world applications such as text categorization, image retrieval, gene expression analysis and more. Here are a few examples:

1. **Text Categorization:** With vast amounts of online data dimensionality reduction helps classify text documents into predefined categories by reducing the feature space like word or phrase features while maintaining accuracy.
2. **Image Retrieval**: As image data grows indexing based on visual content like color, texture, shape rather than just text descriptions has become essential. This allows for better retrieval of images from large databases.
3. **Gene Expression Analysis**: Dimensionality reduction accelerates gene expression analysis help to classify samples like leukemia by identifying key features, improve both speed and accuracy.
4. **Intrusion Detection**: In cybersecurity dimensionality reduction helps analyze user activity patterns to detect suspicious behaviors and intrusions by identifying optimal features for network monitoring.

- **Faster Computation**: With fewer features machine learning algorithms can process data more quickly. This results in faster model training and testing which is particularly useful when working with large datasets.
- **Better Visualization**: As we saw in the earlier figure reducing dimensions makes it easier to visualize data and reveal hidden patterns.
- **Prevent Overfitting**: With few features models are less likely to memorize the training data and overfit. This helps the model generalize better to new, unseen data improve its ability to make accurate predictions.

Disadvantages

- **Data Loss & Reduced Accuracy:** Some important information may be lost during dimensionality reduction and affect model performance.
- **Choosing the Right Components:** Deciding how many dimensions to keep is difficult as keeping too few may lose valuable information while keeping too many can led to overfitting.

Principal Component Analysis

How Principal Component Analysis Works

PCA uses linear algebra to transform data into new features called principal components. It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix. PCA selects the top components with the highest eigenvalues and projects the data onto them simplify the dataset.

> ***Note:*** *It prioritizes the directions where the data varies the most because more variation = more useful information.*

Imagine you're looking at a messy cloud of data points like stars in the sky and want to simplify it. PCA helps you find the "most important angles" to view this cloud so you don't miss the big patterns. Here's how it works step by step:

Step 1: Standardize the Data

Different features may have different units and scales like salary vs. age. To compare them fairly PCA first standardizes the data by making each feature have:

- A mean of 0
- A standard deviation of 1

$$Z = \frac{X - \mu}{\sigma}$$

where:

- μ is the mean of independent features $\mu = \{\mu_1, \mu_2, \cdots, \mu_m\}$
- σ is the standard deviation of independent features $\sigma = \{\sigma_1, \sigma_2, \cdots, \sigma_m\}$

Step 2: Calculate Covariance Matrix

Next PCA calculates the covariance matrix to see how features relate to each other whether they increase or decrease together. The covariance between two features x_1 and x_2 is:

$$cov(x1, x2) = \frac{\sum_{i=1}^{n}(x1_i - \bar{x1})(x2_i - \bar{x2})}{n-1}$$

Where:

- $\bar{x}_1 \; and \; \bar{x}_2$ are the mean values of features $x_1 \; and \; x_2$
- n is the number of data points

The value of covariance can be positive, negative or zeros.

Step 3: Find the Principal Components

PCA identifies new axes where the data spreads out the most:

- **1st Principal Component (PC1):** The direction of maximum variance (most spread).
- **2nd Principal Component (PC2):** The next best direction, *perpendicular to PC1* and so on.

These directions come from the eigenvectors of the covariance matrix and their importance is measured by eigenvalues. For a square matrix A an eigenvector X (a non-zero vector) and its corresponding eigenvalue λ satisfy:

$$AX = \lambda X$$

This means:

- When *A* acts on X it only stretches or shrinks X by the scalar λ.
- The direction of X remains unchanged hence eigenvectors define "stable directions" of A.

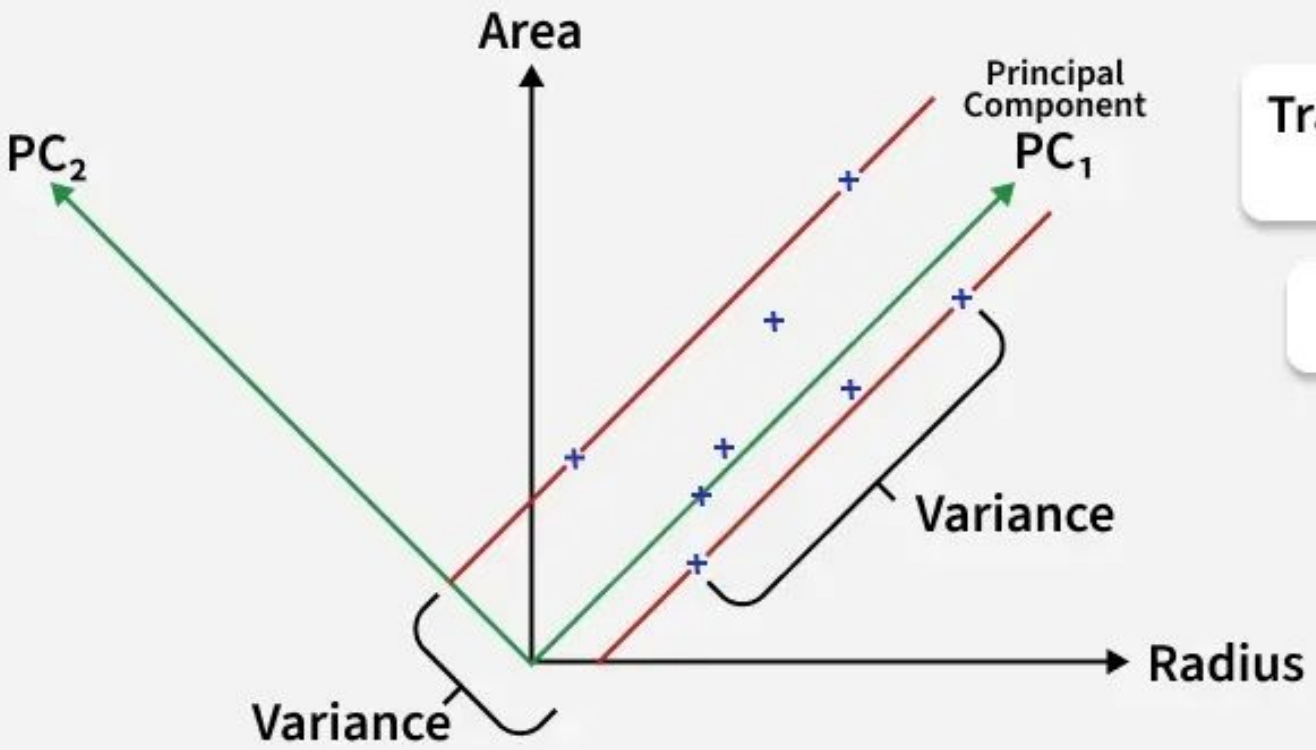
Eigenvalues help rank these directions by importance.

Step 4: Pick the Top Directions & Transform Data

After calculating the eigenvalues and eigenvectors PCA ranks them by the amount of information they capture. We then:

1. Select the top k components that capture most of the variance like 95%.
2. Transform the original dataset by projecting it onto these top components.

This means we reduce the number of features (dimensions) while keeping the important patterns in the data.



Transformation 2D-1D

$PC_1 > PC2$

In the above image the original dataset has two features "Radius" and "Area" represented by the black axes. PCA identifies two new directions: PC_1 and PC_2 which are the principal components.

- These new axes are rotated versions of the original ones. PC_1 captures the maximum variance in the data meaning it holds the most information while PC_2 captures the remaining variance and is perpendicular to PC_1.
- The spread of data is much wider along PC_1 than along PC_2. This is why PC_1 is chosen for dimensionality reduction. By projecting the data points (blue crosses) onto PC_1 we effectively transform the 2D data into 1D and retain most of the important structure and patterns.

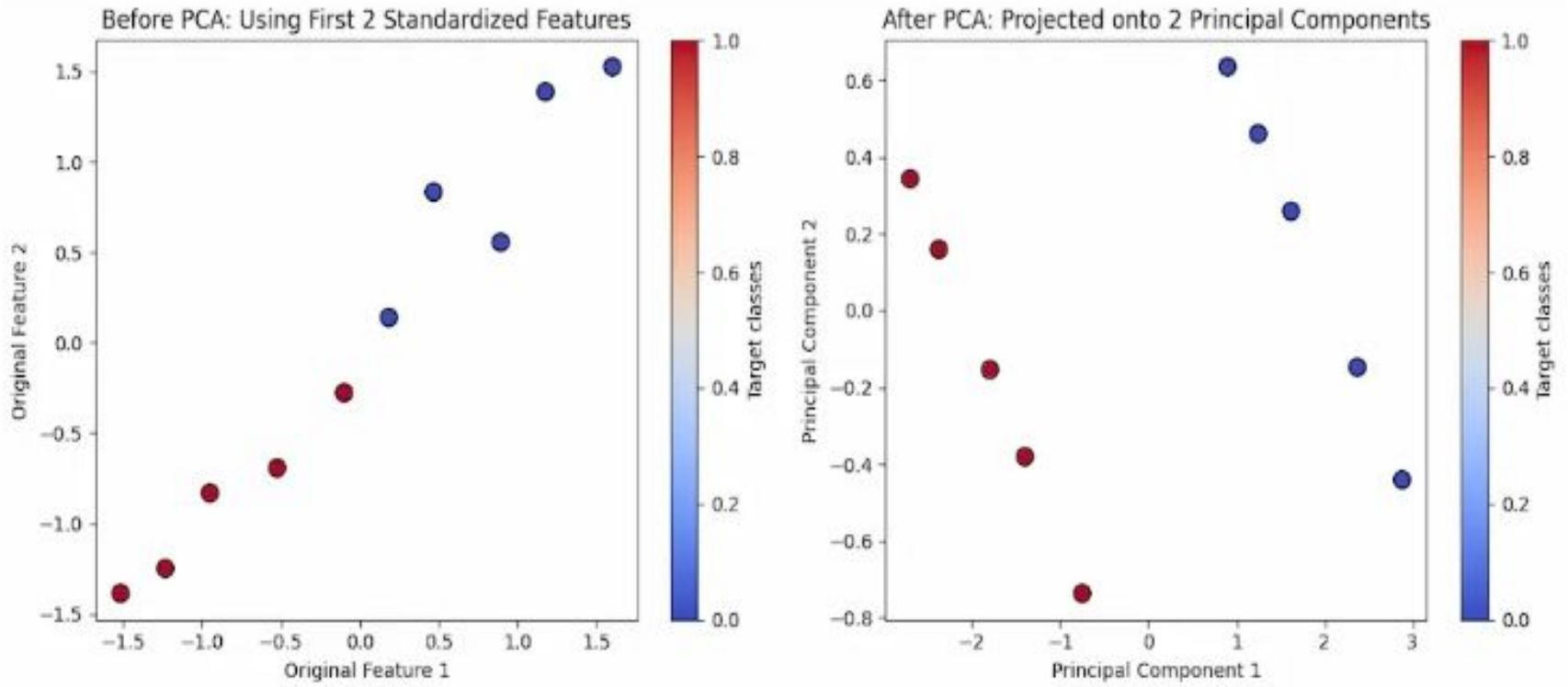
Advantages of Principal Component Analysis

1. **Multicollinearity Handling:** Creates new, uncorrelated variables to address issues when original features are highly correlated.
2. **Noise Reduction:** Eliminates components with low variance enhance data clarity.
3. **Data Compression:** Represents data with fewer components reduce storage needs and speeding up processing.
4. **Outlier Detection:** Identifies unusual data points by showing which ones deviate significantly in the reduced space.

Disadvantages of Principal Component Analysis

1. **Interpretation Challenges:** The new components are combinations of original variables which can be hard to explain.
2. **Data Scaling Sensitivity:** Requires proper scaling of data before application or results may be misleading.
3. **Information Loss:** Reducing dimensions may lose some important information if too few components are kept.
4. **Assumption of Linearity:** Works best when relationships between variables are linear and may struggle with non-linear data.
5. **Computational Complexity:** Can be slow and resource-intensive on very large datasets.
6. **Risk of Overfitting:** Using too many components or working with a small dataset might lead to models that don't generalize well.

Output:



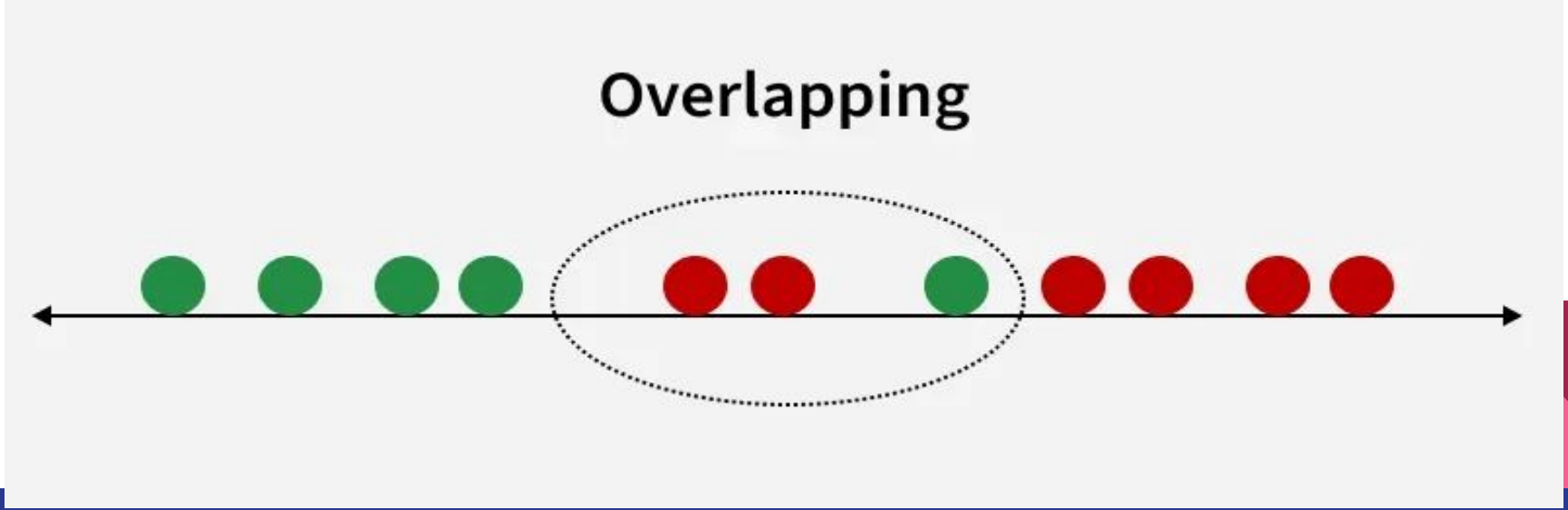
PCA Algorithm

- **Left Plot Before PCA**: This shows the **original standardized data** plotted using the first two features. There is **no guarantee of clear separation** between classes as these are raw input dimensions.
- **Right Plot After PCA**: This displays the **transformed data** using the **top 2 principal components**. These new components capture the **maximum variance** often showing better **class separation and structure** making it easier to analyze or model.

Linear Discriminant Analysis in Machine Learning

Linear Discriminant Analysis in Machine Learning

Linear Discriminant Analysis (LDA) also known as Normal Discriminant Analysis is supervised classification problem that helps separate two or more classes by converting higher-dimensional data space into a lower-dimensional space. It is used to identify a linear combination of features that best separates classes within a dataset.



For example we have two classes that need to be separated efficiently. Each class may have multiple features and using a single feature to classify them may result in overlapping. To solve this LDA is used as it uses multiple features to improve classification accuracy. LDA works by some assumptions and we are required to understand them so that we have a better understanding of its working.

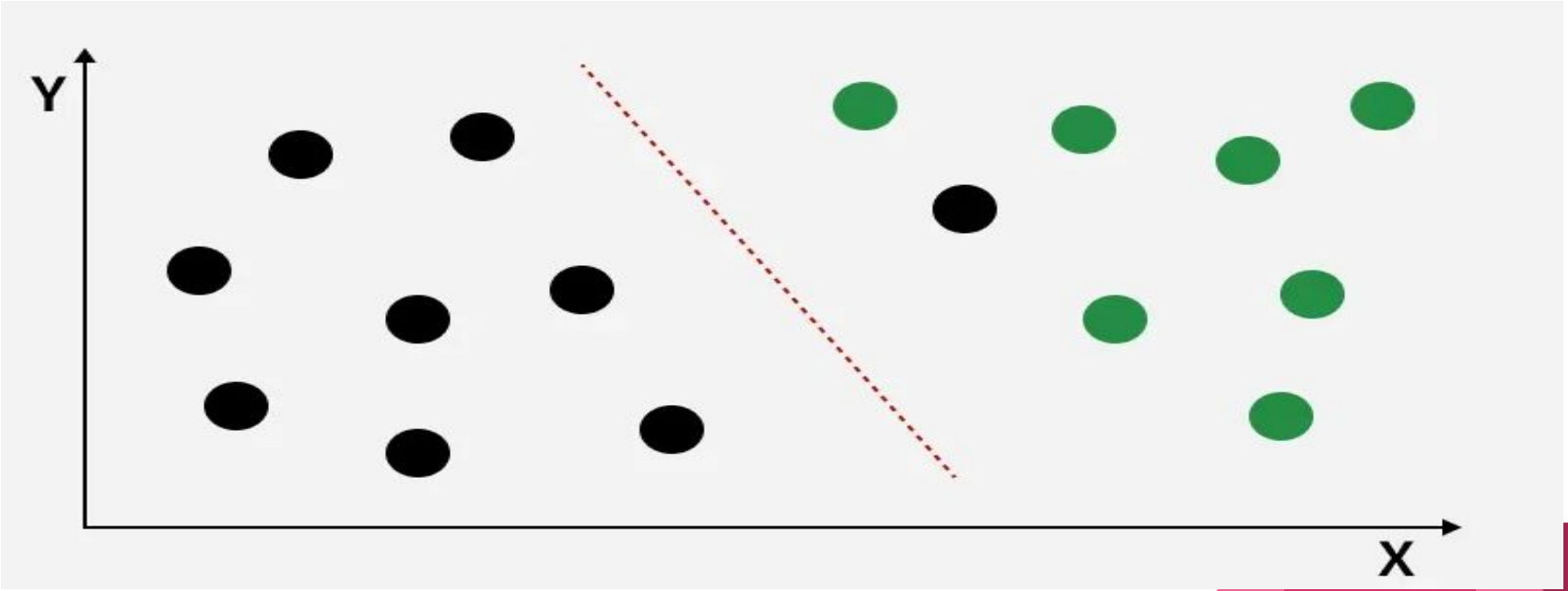
Key Assumptions of LDA

For LDA to perform effectively, certain assumptions are made:

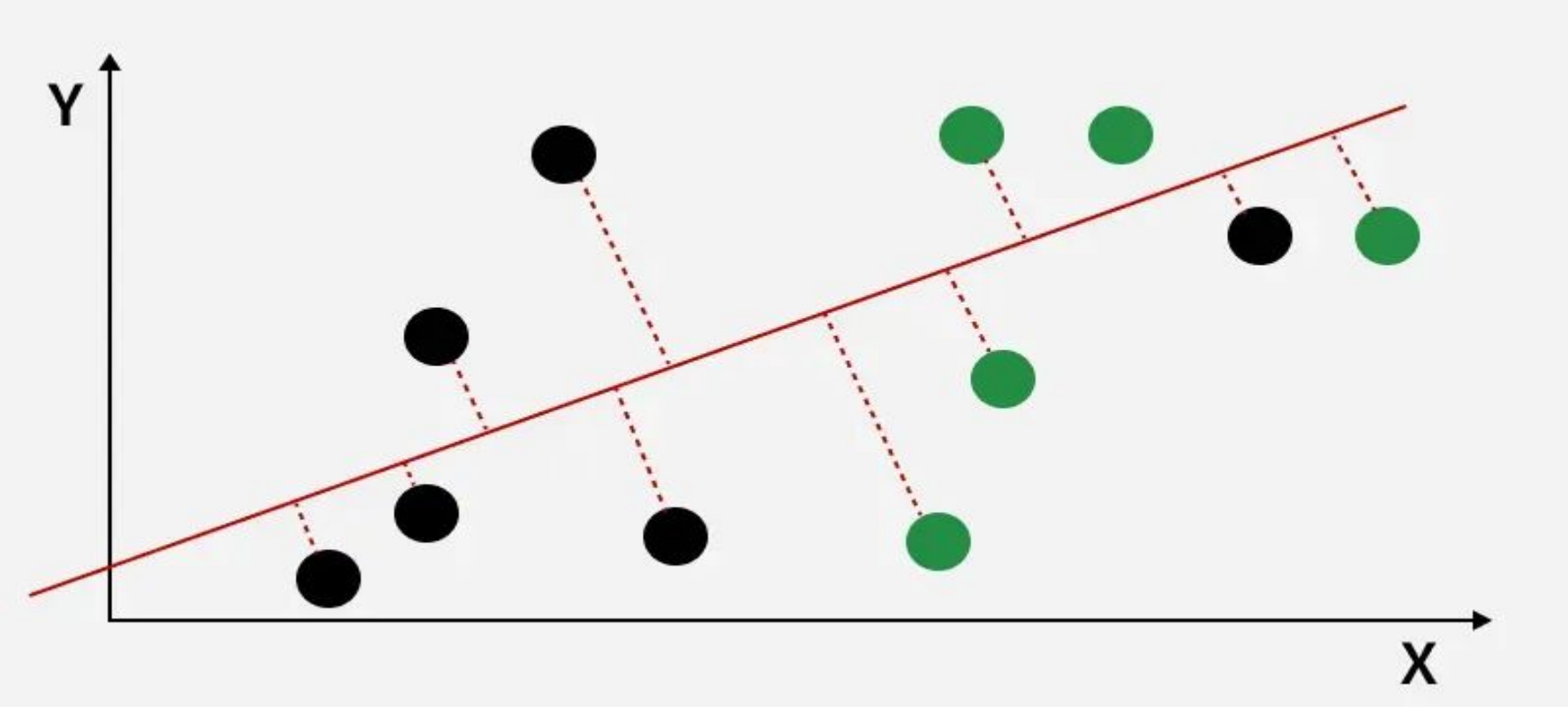
- **Gaussian Distribution:** The data in each class should follow a normal bell-shaped distribution.
- **Equal Covariance Matrices:** All classes should have the same covariance structure.
- **Linear Separability:** The data should be separable using a straight line or plane.

LDA can produce very good results if it meets these assumptions. For example when data points belonging to two classes are plotted, if they are not linearly separable LDA will attempt to find a projection that maximizes class separability.

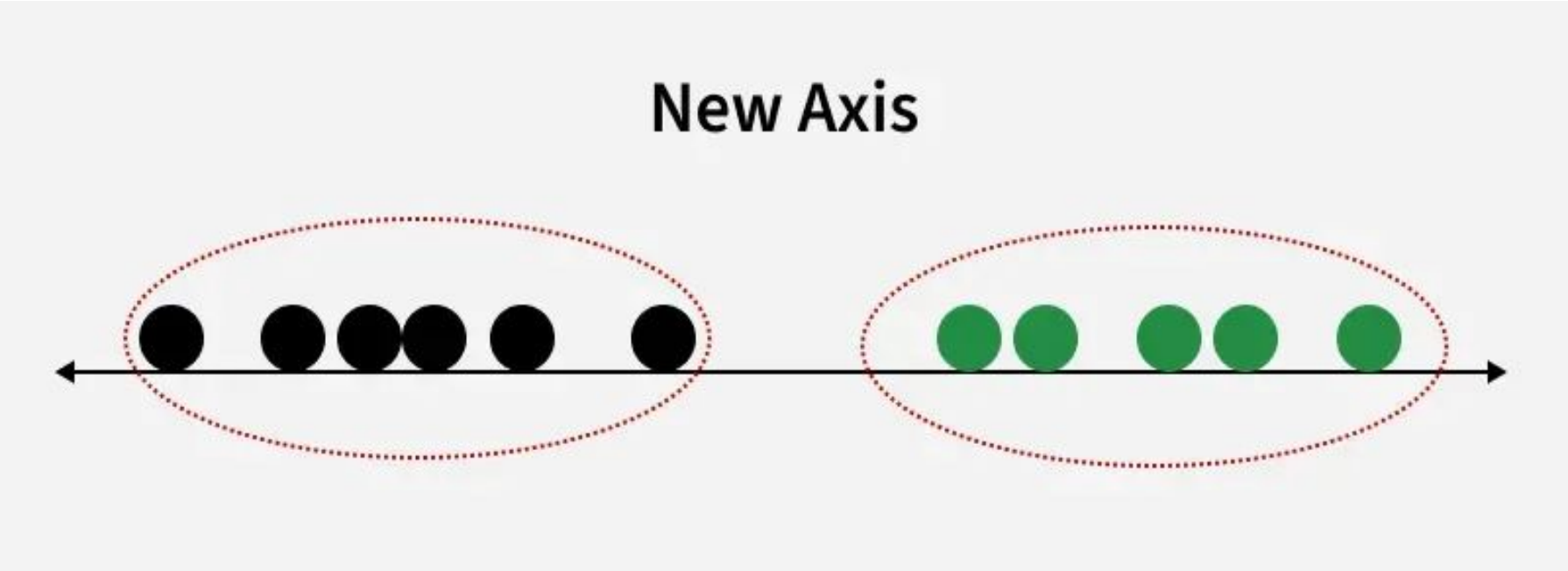
The image shows classes (black and green) that are not linearly separable. LDA finds a new axis (red dashed line) that maximizes the distance between class means while minimizing within-class variance, improving class separation for better classification.



The perpendicular distance from the decision boundary to the data points shows how LDA reduces within-class variation and increases class separability. The data points are then projected onto the new axis, as shown in the figure below.



This shows how LDA creates a new axis to project the data and separate two classes along a linear path. However, when class distributions share the same mean, LDA cannot find a separating axis and non-linear discriminant analysis is needed.



How LDA work

LDA works by finding directions in the feature space that best separate the classes. It does this by maximizing the difference between the class means while minimizing the spread within each class.

Let's assume we have two classes with d-dimensional samples such as $x_1, x_2, \ldots x_n$ where:

- n_1 samples belong to class c_1
- n_2 samples belong to class c_2.

If x_i represents a data point its projection onto the line represented by the unit vector v is $v^T x_i$. Let the means of class c_1 and class c_2 before projection be μ_1 and μ_2 respectively. After projection the new means are $\hat{\mu}_1 = v^T \mu_1$ and $\hat{\mu}_2 = v^T \mu_2$.

Our aim to normalize the difference $|\hat{\mu}_1 - \hat{\mu}_2|$ to maximize the class separation. The scatter for samples of class c_1 is calculated as:

$$s_1^2 = \sum_{x_i \in c_1} (x_i - \mu_1)^2$$

Similarly for class c_2:

$$s_2^2 = \sum_{x_i \in c_2} (x_i - \mu_2)^2$$

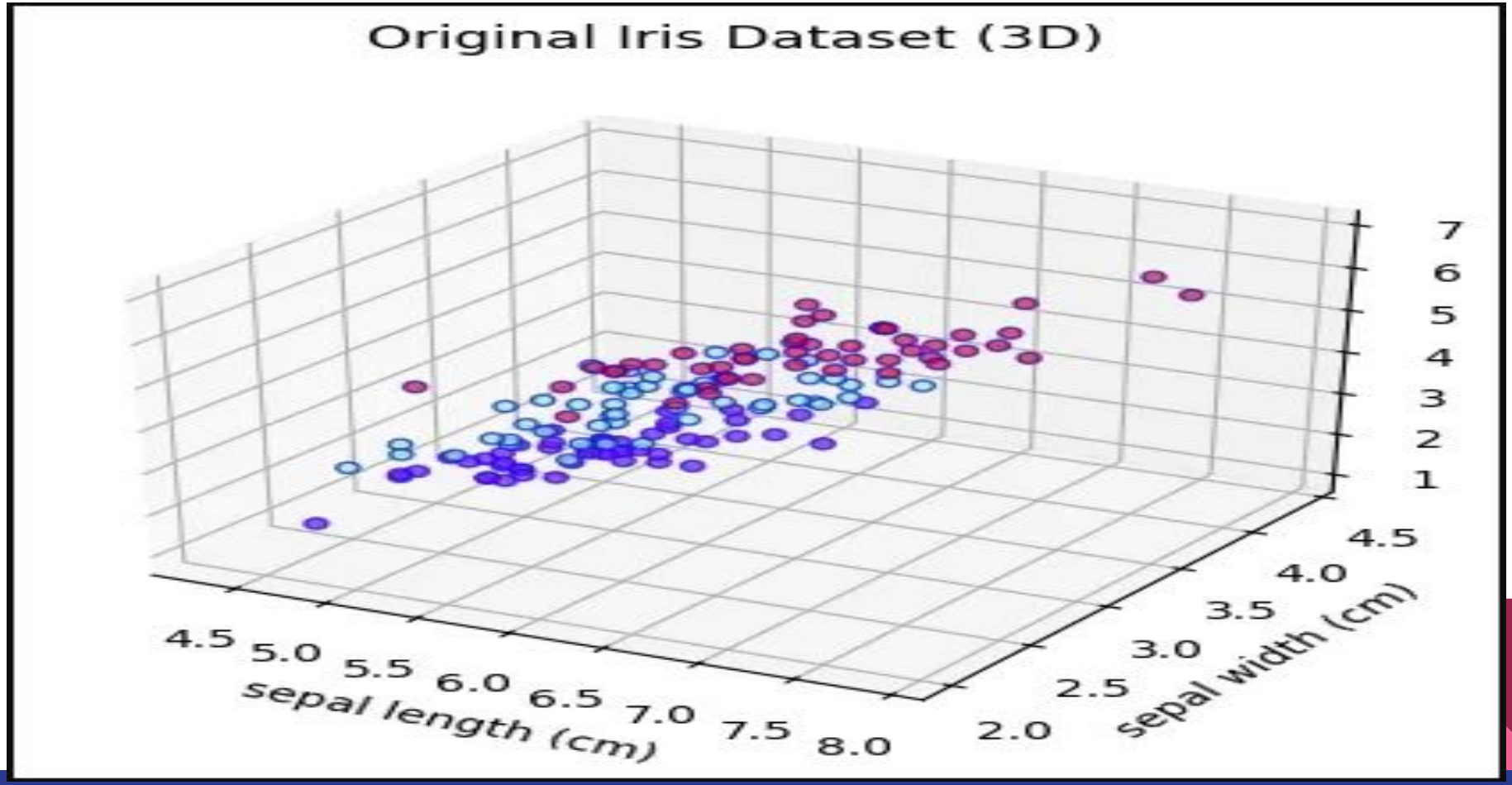
The goal is to maximize the ratio of the between-class scatter to the within-class scatter, which leads us to the following criteria:

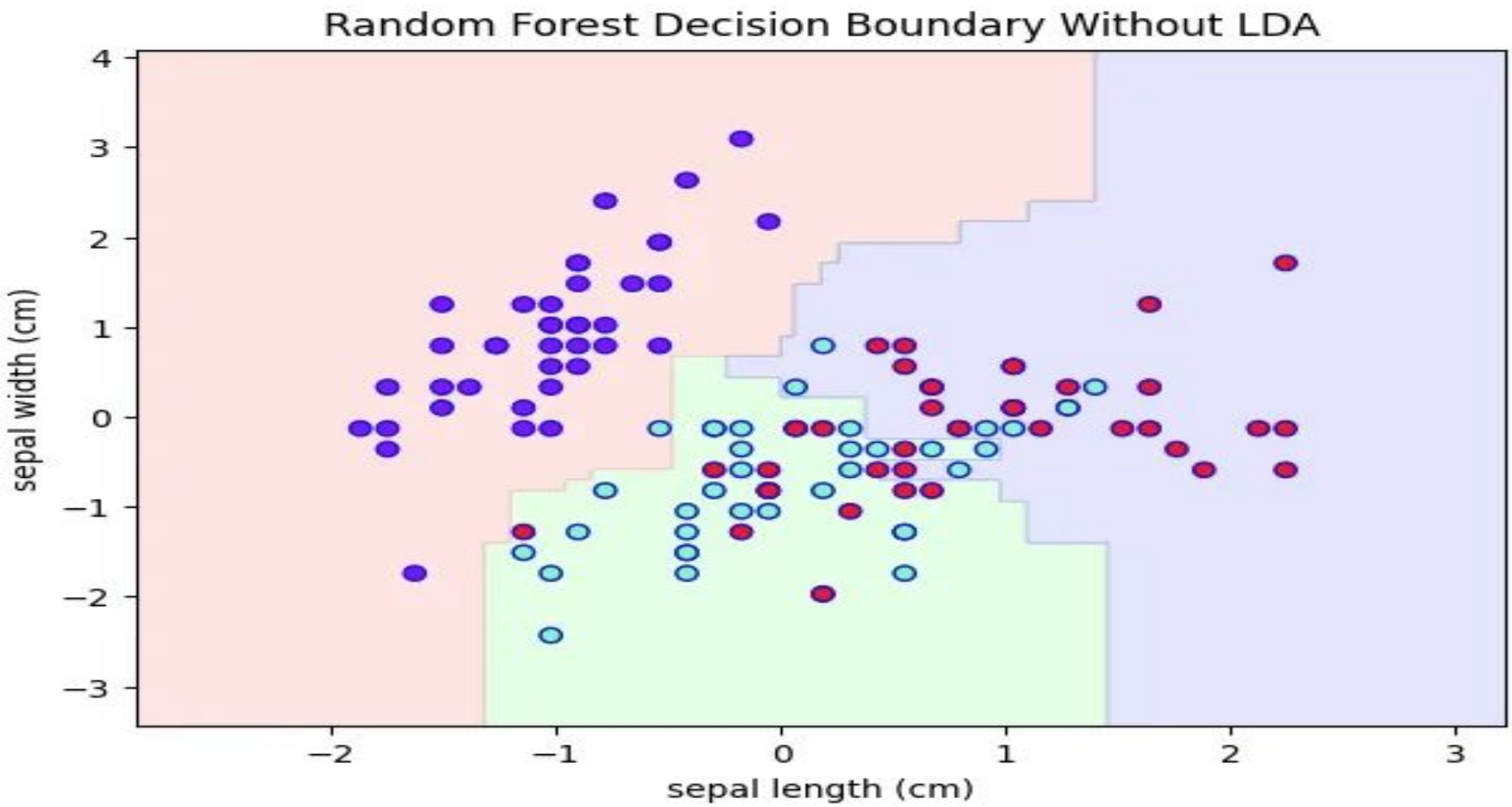
$$J(v) = \frac{|\hat{\mu}_1 - \hat{\mu}_2|}{s_1^2 + s_2^2}$$

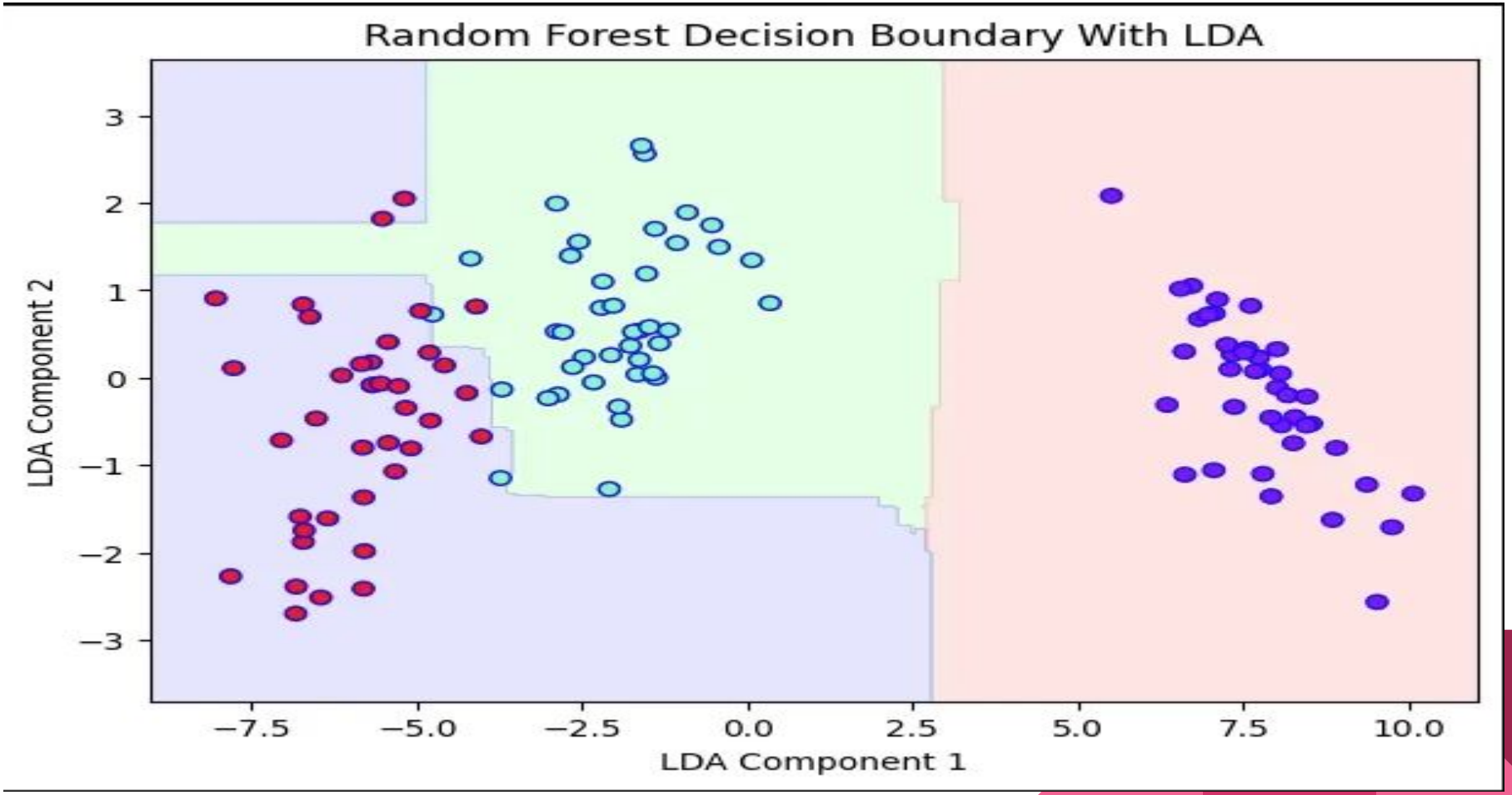
For the best separation we calculate the eigenvector corresponding to the highest eigenvalue of the scatter matrices $s_w^{-1} s_b$.

Extensions to LDA

1. **Quadratic Discriminant Analysis (QDA):** Each class uses its own estimate of variance (or covariance) allowing it to handle more complex relationships.
2. **Flexible Discriminant Analysis (FDA):** Uses non-linear combinations of inputs such as splines to handle non-linear separability.
3. **Regularized Discriminant Analysis (RDA):** Introduces regularization into the covariance estimate to prevent overfitting.


Original Iris Dataset (3D)
7
6
5
4
3
2
1
4.5
4.0
3.5
3.0
2.5
2.0
sepal width (cm)
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
sepal length (cm)


Random Forest Decision Boundary Without LDA
sepal width (cm)
sepal length (cm)
4
3
2
1
0
−1
−2
−3
−2
−1
0
1
2
3


Random Forest Decision Boundary With LDA
LDA Component 2
LDA Component 1
3
2
1
0
−1
−2
−3
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

Advantages of LDA

- Simple and computationally efficient.
- Works well even when the number of features is much larger than the number of training samples.
- Can handle multicollinearity.

Disadvantages of LDA

- Assumes Gaussian distribution of data which may not always be the case.
- Assumes equal covariance matrices for different classes which may not hold in all datasets.
- Assumes linear separability which is not always true.
- May not always perform well in high-dimensional feature spaces.

Applications of LDA

1. **Face Recognition**: It is used to reduce the high-dimensional feature space of pixel values in face recognition applications helping to identify faces more efficiently.
2. **Medical Diagnosis**: It classifies disease severity in mild, moderate or severe based on patient parameters helping in decision-making for treatment.
3. **Customer Identification**: It can help identify customer segments most likely to purchase a specific product based on survey data.

LDA focuses on maximizing the ratio of between-class scatter to within-class scatter, making it effective for classification tasks where class separability is crucial. It can handle multi-class classification problems and is less sensitive to outliers compared to PCA.

PCA aims to capture the maximum variance in the data by finding orthogonal directions (principal components). It is useful for reducing the dimensionality of high-dimensional datasets, improving visualization, and reducing computational complexity. However, PCA does not consider class labels and is more sensitive to outliers.

- **LDA** requires class labels and is therefore a supervised technique, making it more suitable for classification tasks.
- **PCA** does not require class labels and is an unsupervised technique, making it more suitable for exploratory data analysis and feature extraction.
- **LDA** is less sensitive to outliers compared to PCA.
- **PCA** can handle high-dimensional datasets efficiently and is widely used for data visualization.

Attribute	LDA	PCA
Definition	Latent Dirichlet Allocation is a probabilistic model used for topic modeling.	Principal Component Analysis is a statistical technique used for dimensionality reduction.
Input	Text documents or any collection of discrete data.	Numerical data.
Output	Topic distribution for each document.	Transformed data with reduced dimensions.
Goal	To discover latent topics within a document collection.	To find the most important features or patterns in the data.
Technique	Generative model based on probability distributions.	Linear transformation based on eigenvectors.
Application	Text mining, document classification, information retrieval.	Data visualization, noise reduction, feature extraction.
Interpretability	Topics can be interpreted by analyzing the most probable words.	Components may not have direct interpretability.
Supervised/Unsupervised	Unsupervised learning technique.	Unsupervised learning technique.
Dimensionality Reduction	Does not inherently reduce dimensions.	Reduces dimensions by projecting data onto new axes.